

Lab02 – SLURM part 1

Introduction

The objective of this exercise is to run a Jupyter notebook as an interactive SLURM job on the Ares or Athena cluster.

Useful links:

1. [Ares preliminary access essentials - Maciej Pawlik - Confluence \(cyfronet.pl\)](#)
2. [Athena - Maciej Pawlik - Confluence \(cyfronet.pl\)](#)
3. [Slurm Workload Manager - Quick Start User Guide \(schedmd.com\)](#)
4. [Ustawianie środowiska aplikacji - Modules - PLGrid Dokumentacja - Confluence \(cyfronet.pl\)](#)
(note that “plgrid” suffix is not used on ares!)

SLURM jobs

A SLURM job is created from a shell script that typically has the following structure:

```
#!/bin/bash

# submit options
#SBATCH --ntasks=1
#SBATCH --time=00:05:00

# initialization
module load package/version

# data handling and work
cd /path/to/working/directory
binary [arguments]

exit
```

The job is submitted using the **sbatch** command.

A SLURM job is a collection of distributed **tasks**. A task exists on a single node, but may consist of multiple processes (e.g. forked).

A job may also consist of multiple **steps** which can run sequentially or in parallel. Steps are specified using **srun** command inside SLURM. An example SLURM job with steps is shown in Fig. 1.

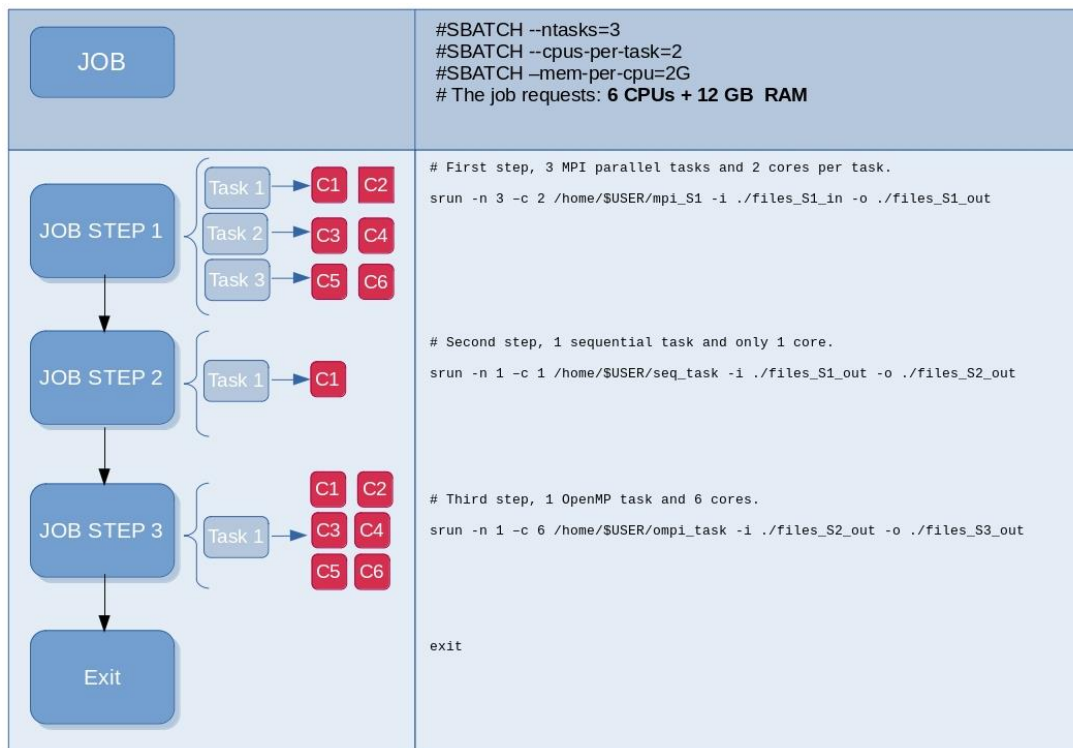
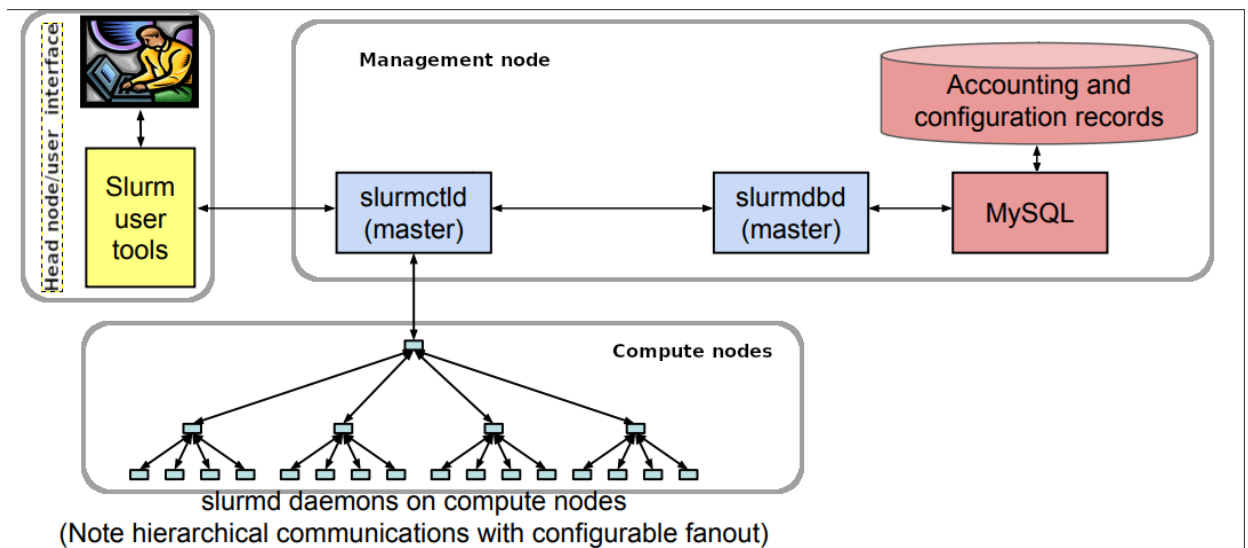
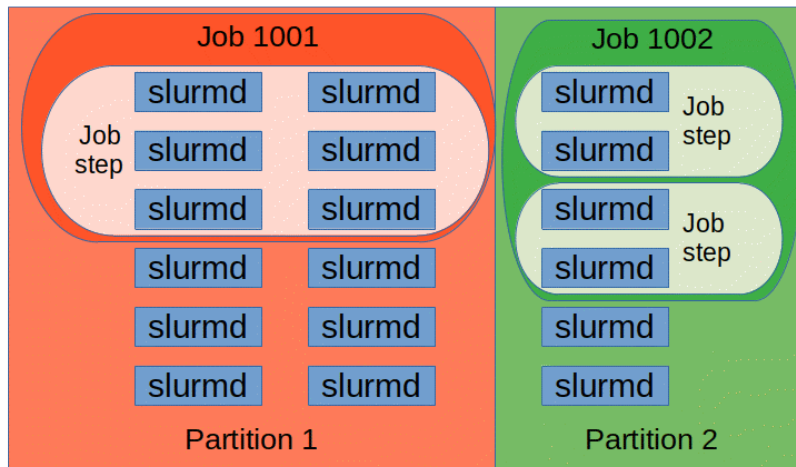


Figure 1: Example SLURM job structure and script. (Source: https://garnatxadoc.uv.es/slurm/slurm_info.html)

SLURM architecture



SLURM entities



SLURM commands

Basic commands

- `sinfo` – info about nodes
- `squeue` – queue status
- `sbatch` – submit a job
- `srun` – run an interactive job/job step
- `scancel` – cancel job
- `sacct` – accounting information

Examples:

```
$ sinfo -p plgrid
```

```
$ squeue
```

```
$ sbatch job.sh
```

```
$ srun -N 1 --ntasks-per-node=1 -p plgrid-testing --pty /bin/bash -l
```

```
$ scancel <jobid>
```

```
$ sacct -X
```

Resource specification

- How to specify resources? With `-N`, `-n`, `--ntasks-per-node` etc. man `sbatch`
- `sbatch -N 2` – Node count
- `sbatch -N 1 --ntasks-per-node=6` – Node count with 6 tasks per node = 6 cores
- `sbatch -N 2 --ntasks-per-node=6` – Node count with 6 tasks per node = 12 cores on 2 nodes
- `sbatch -N 2 --ntasks-per-node=6 -t 20:00` – declared duration of job (HH:MM:SS), defaults to 15 minutes
- `sbatch -A plglscclclass23-cpu` – declare using the `plglscclclass23-cpu` grant for your computations, this is important if you have other computing grants and want to use a specific one!
- The account name `plglscclclass23-cpu` can be determined by inspecting the ``hpc-grants`` output, and looking for “allocation” name. Please note that there can be multiple grants, use the most recent one!
- Be cautious about available node configurations, you are responsible for “fitting in”

- In most cases jobs are specified with “job scripts”
- Example job script (*.sh file), parameters can be included in the job script or be supplied from the command line as sbatch arguments:

```
#!/bin/bash
#SBATCH -p plgrid
#SBATCH -N 1
#SBATCH --cpus-per-task=4
# the above lines are read and interpreted by sbatch, this one and later are not

# command, this will be executed on a compute node
sleep 1m
```

Interactive work

- Typical use cases?

```
$ srun -p plgrid-now --time 1:00:00 --pty /bin/bash -l
```

- Hint: using the plgrid-now partition usually leads to very short queue time with the cost of imposing limits on the job like shorter runtime
- Hint2: using the plgrid-testing partition also yields shorter queue times with less severe restrictions

Environment

- SLURM communicates a lot of information through environment variables
- \$SLURM_SUBMIT_HOST - host, what is the host from which the job originated?
- \$SLURM_SUBMIT_DIR – directory, from where the job was submitted
- Other variables: man sbatch, or check with ‘env’ command in an interactive job

Array/parametric jobs

- We can create multiple instances of the same job by using the –array parameter.

```
$ sbatch -N 1 --ntasks-per-node=1 -p plgrid-testing --array=1-4 test.sh
```

- This will create an array job composed of 4 jobs. This can be verified by using the squeue command. It’s possible to “know” which job is which by reading the SLURM_ARRAY_TASK_ID environment variable:

```
$ echo $SLURM_ARRAY_TASK_ID
```

Modules

- Managing available software is done through the system called “modules”.
- The software includes: scientific software, libraries (e.g. sqlite, cuda), core utilities (compiles, math libraries)
- The “module” command is used to work with the system, where subcommands include:
 - avail – see the list of available modules
 - load – to load a module
 - list – to list loaded modules
 - purge – clean your environment from any loaded modules
 - help – consult the manual

- Some examples:
\$ module avail
(..This will list all modules..)
\$ module load plumed
(..This will load the plumed module..)
\$ module purge
(..This will list the unloaded modules..)
- What does the module command actually do? It doesn't install the software, it adjusts the environment, so the application becomes available as it would be installed in a usual way. You can think of a module as a preset of PATH, LD_LIBRARY_PATH and other important environment variables.
- Load modules in the job script!

Exercise - running Jupyter on a supercomputer - steps

On Ares/Athena:

1. Login to Ares or Athena (the lab was tested on Ares)
2. Run an interactive job using command:
srun --time=2:00:00 --mem=8G --ntasks 1 --gres=gpu:1 --partition=plgrid-gpu-v100 --account=plglscclclass24-gpu --pty /bin/bash

You will see output similar to:

srun: job 11818449 queued and waiting for resources

srun: job 11818449 has been allocated resources

[ares][plgbalis@ag0009 ~]\$

This interactive job provides a shell access to a worker node, in this case **ag0009**.

3. Activate a python virtual environment prepared on the shared group storage:

conda activate \$PLG_GROUPS_STORAGE/plgglscclass/.conda/envs/lscslab

Notes:

- It is better to configure your application (e.g. install packages) on a worker node using an interactive job, rather than on the login node
- If conda is not available, load module using command **module load miniconda3**
- Look at installed packages using **conda list**
- To install your own packages, you can use your **\$HOME** disk, but the quota is limited.
- You can also use your **\$SCRATCH** storage space. To do this, you can configure conda:

conda config --add envs_dirs \${SCRATCH}/.conda/envs

conda config --add pkgs_dirs \${SCRATCH}/.conda/pkgs

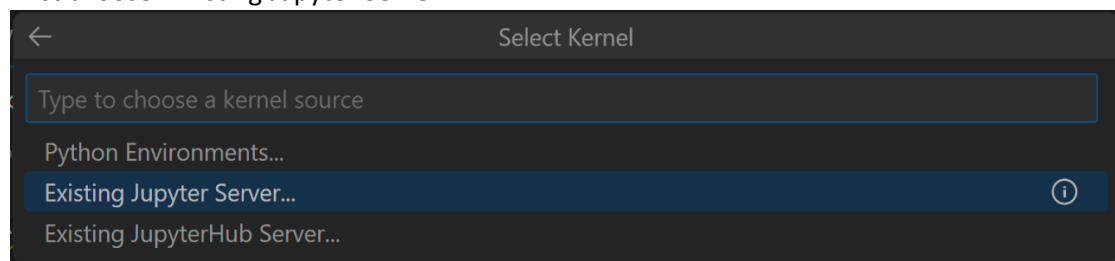
4. Copy example notebook to your home directory:
cp \$PLG_GROUPS_STORAGE/plgglscclass/lscslab/lsc_lab02.ipynb \$HOME
5. Start jupyter (provide your node name to **--ip** parameter):
jupyter --notebook --no-browser --port=8888 --ip=<node_name>

Note the connection URL, it will be needed later!

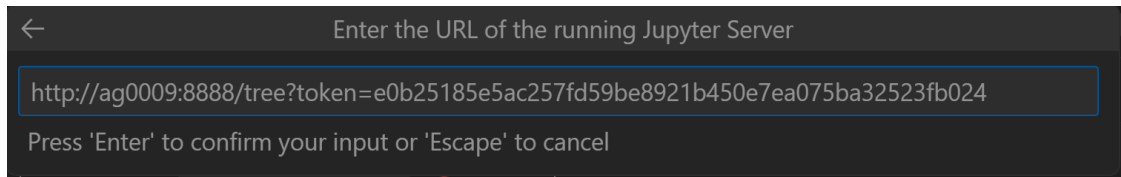
On your machine:

6. Start Visual Studio Code and install extension **Remote – SSH** from Microsoft.
7. Start a remote SSH connection to Ares/Athena (you can invoke command palette **Ctrl+Shift+P** and search for **Remote-SSH: Connect to Host**)
8. Open the **lsc_lab02.ipynb** file
9. Connect to the Jupyter server started on the worker node.

First choose “Existing Jupyter Server”



Then paste the Jupyter connection URL printed earlier. For example:

A terminal window with a dark background. At the top, it says "Enter the URL of the running Jupyter Server". Below this, a text input field contains the URL "http://ag0009:8888/tree?token=e0b25185e5ac257fd59be8921b450e7ea075ba32523fb024". Below the input field, it says "Press 'Enter' to confirm your input or 'Escape' to cancel".

← Enter the URL of the running Jupyter Server

http://ag0009:8888/tree?token=e0b25185e5ac257fd59be8921b450e7ea075ba32523fb024

Press 'Enter' to confirm your input or 'Escape' to cancel

10. Run the example code and see if it works!

Assignment

(5 pts)

Perform data analysis/exploration/visualization/ML using Jupyter/Pandas/Matplotlib on the **Yelp dataset**, available in the shared group storage:

\$PLG_GROUPS_STORAGE/plgglscclclass/yelp_dataset

Configure an environment (python venv) on your \$SCRATCH storage.

The dataset was downloaded from [Yelp Dataset \(kaggle.com\)](https://www.kaggle.com/yelp-dataset). You can use notebooks available there.

Grading:

- Configuration of the environment: 2p
- Quality of data analysis: 3p